TI-2600 Laboratório de Desenvolvimento Multiplataforma

Antonio Carvalho - Treinamentos

Jetpack Compose

Jetpack Compose

- Compose é uma biblioteca da suíte Jetpack que permite a criação de telas utilizando composição, sem a necessidade de declarar arquivos XML

Jetpack Compose

- Desta forma é possível criar telas para aplicativos Android com Kotlin utilizando duas maneiras
- **Imperativa**, preocupa-se em definir como a tela será mostrada, esta maneira embora mais rápida, pode resultar em atividades redundantes para o processador e pouco reuso. As telas criadas com arquivos XML são feitas de maneira Imperativa
- **Declarativa**, preocupa-se em definir o que deve ser mostrado na tela, esta maneira pode resultar em um pouco menos de performance e maior dificuldade para *debug*, porém facilita a manutenibilidade do código e o seu reuso

Jetpack Compose

- Para utilizar o **Compose** é preciso criar uma função que invoque os componentes **Composable**. Isto fará com que a função também seja **Composable**.
- As funções do tipo **Composable** precisam ter uma anotação **@Composable** e devem seguir algumas regras como:
  - Não deve retornar nada
  - Deve ser rápida, idempotente e sem efeitos colaterais
  - Ao ser invocada com os mesmos parâmetros deve produzir o mesmo resultado, portanto não deve utilizar chamadas de métodos ***random*** ou ***variáveis globais***

Jetpack Compose – **Text**

Text Permite a exibição de um texto estático

Propriedades necessárias:

- **text** : Deve conter o texto a ser exibido

Propriedade principais:

- **color** : Define a cor do texto
- **maxLines** : Limita o número de linhas de texto a ser exibida
- **overflow** : Controla como o texto será tratado ao exceder o espaço

Exemplo:

```
Text(  text = "Hello World!",
       color = Color.Blue,
       maxLines = 1,
       overflow = TextOverflow.Ellipsis  )
```

Jetpack Compose – **Button**

Button Permite que o usuário clique, e pode executar uma ação ao ser clicado

Propriedades necessárias:

- **onClick** : Deve conter o código a ser executado ao ser clicado

Propriedade principais:

- **enable**: Determina se o botão está habilitado ou desabilitado.
- **contentPadding** : Define o espaçamento interno do conteúdo
- **colors** : Personaliza cores de fundo e texto, usando **ButtonDefaults.buttonColors()**
- **content** : Especifica o que estará contido dentro do Botão

Exemplo:

```
Button(
    onClick = { println("Botão clicado!") },
    enabled = true
) {
    Text("Clique aqui")
}
```

Jetpack Compose – **TextField**

TextField Permite que o usuário possa preencher um texto dentro do campo, que poderá ser guardado em uma variável de estado

Propriedades necessárias:

- **value** : Texto que será mostrado no corpo do componente
- **onValueChange** : Deve conter o código a ser executado quando houver uma mudança no texto

Propriedade principais:

- **label**: Informa o rótulo a ser exibido acima do campo.
- **placeholder** : Define o texto de dica que aparecerá quando não houver nada digitado no campo.
- **keyboardOptions** : Configurações de teclado, como tipo de entrada.
- **singleLine** : Determina se o campo é limitado a uma única linha.

Exemplo:

```
var text by remember { mutableStateOf("") }
TextField(
    value = text,
    onValueChange = { text = it },
    label = { Text("Seu nome") },
    singleLine = true
)
```

Jetpack Compose – **Column**

Column permite organizar os elementos internos em um **Container**, de maneira vertical, onde os elementos são posicionados um abaixo do outro

Propriedade principais:

- **horizontalAlignment**: Define o formato do alinhamento horizontal dos componentes podendo ser Start, End, Center ou Stretch
- **verticalArrangement** : Define o formato do arranjo vertcal dos componentes podendo ser Start, End, Center, SpaceBetween, SpaceAround, SpaceEvenly
- **modifier** : Permite ajustes no Layout como padding, backgroundColor, border, etc..

Exemplo:

```
Column(
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    Text("Item 1")
    Text("Item 2")
}
```

Jetpack Compose – **Row**

Row permite organizar os elementos internos em um **Container**, de maneira horizontal, onde os elementos são posicionados um ao lado do outro

Propriedade principais:

- **verticalAlignment**: Define o formato do alinhamento vertical dos componentes podendo ser Start, End, Center ou Stretch
- **horizontalArrangement** : Define o formato do arranjo horizontal dos componentes podendo ser Start, End, Center, SpaceBetween, SpaceAround, SpaceEvenly
- **modifier** : Permite ajustes no Layout como padding, backgroundColor, border, etc..

Exemplo:

```
Row(
    horizontalArrangement = Arrangement.spacedBy(16.dp)
) {
    Text("Esquerda")
    Text("Direita")
}
```

Jetpack Compose – **Box**

Box permite organizar os elementos internos em um **Container**, empilhando-os uns sobre os outros

Propriedade principais:

- **contentAlignment**: Define o formato do alinhamento dentro do Box
- **modifier** : Permite ajustes no Layout como padding, backgroundColor, border, etc..

Exemplo:

```
Box(
    contentAlignment = Alignment.Center,
    modifier = Modifier.size(100.dp)
) {
    Text("Centralizado")
}
```

Jetpack Compose – **Modifier**

Modifier é usado para personalizar e estilizar componentes de UI no Jetpack Compose. Ele permite ajustar layouts, adicionar comportamentos e aplicar efeitos visuais.

Dimensões e Layout:

- Definir tamanhos**: size(), fillMaxSize()**
- Ajustar margens e espaçamentos: **padding()**

Interações:

- Detectar toques: **clickable** { }
- Adicionar gestos: **draggable**(), **scrollable**()

Estilização:

- Adicionar bordar: **border**()
- Aplicar fundo: **background**()
- Controlar a transparência: **alpha**()

Alinhamento:

- Centralizar itens: **align**() ou **wrapContent**()

Jetpack Compose – **Modifier**

Exemplo:

```
Text(
    text = "Olá, Modifier!",
    modifier = Modifier
        .padding(16.dp)
        .background(Color.LightGray)
        .border(1.dp, Color.Black)
        .size(200.dp)
)
```

Jetpack Compose – **Modifier**

Encadeamento de Modifiers

Os **modifiers** podem ser encadeados para aplicar várias modificações a um componente

A ordem dos modificadores importa, pois as modificações são aplicadas na sequencia em que aparecem.

Exemplo:

```
Modifier
    .padding(8.dp)
    .background(Color.Blue)
    .clickable { println("Clicado!") }
```

Jetpack Compose – **Scaffold**

Scaffold É um container que organiza a interface de forma consistente, subdividindo nas seguintes regiões principais do Scaffold (top bar, bottom bar, drawer, FAB).

Propriedade principais:

- **topBar**: Define a barra superior da interface (como um `TopAppBar`), usado frequentemente para títulos, ícones de navegação ou ações.
- **bottomBar**: Define a barra inferior da interface (como uma `BottomNavigation`), ideal para menus de navegação ou botões de ações fixos na parte inferior.
- **drawerContent**: Permite adicionar um painel deslizante lateral (Drawer), comumente usado para navegação lateral ou opções adicionais.
- **floattingActionButton**: Botão de ação flutuante (FAB), normalmente utilizado para ações principais, como "Adicionar" ou "Compartilhar".

Jetpack Compose – **Scaffold**

Propriedade principais:

- **content**: Define o conteúdo principal da inteface, este é o espaço onde os componentes principais da tela são exibidos.
- **backgroundColor**: Define a cor de fundo da tela, facilita a adaptação do tema ou estilo do app.
- **modifier**: Propriedade usada para modificar o layout do `Scaffold`, como margem, padding ou outros ajustes.
- **snackbarHost**: Gerencia a exibição de `SnackBars`, que são mensagens transitórias para feedback ao usuário.
- **drawerGesturesEnabled**: Controla se o painel lateral pode ser aberto via gestos.

Jetpack Compose – **Scaffold**

Exemplo:

```
import androidx.compose.material.Scaffold
import androidx.compose.material.TopAppBar
import androidx.compose.material.Text
import androidx.compose.runtime.Composable

@Composable
fun MyApp() {
    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Exemplo Scaffold") })
        },
        content = {
            Text("Olá, mundo!")
        }
    )
}
```